

HR Workflow Designer Module – Case Study Solution

(<https://github.com/1Terin/HR-Workflow-Designer-Module>)

1) Problem Statement

The goal was to build a mini HR Workflow Designer where HR admins can visually design and test internal workflows (onboarding, leave approval, document verification) without backend dependency.

The prototype needed to prove strong frontend architecture, custom node configuration, mock API integration, and workflow simulation capability.

2) Approach and Design Philosophy

This solution was designed with a modular, scalable, and type-safe approach using React + TypeScript.

Core principles followed:

Separation of concerns (canvas, forms, API, store, utilities)

Reusable abstractions for node management and validation

Controlled/dynamic form handling for extensibility

In-memory workflow execution sandbox for rapid feedback

3) Tech Stack and Why

React + Vite + TypeScript: fast setup, strong typing, maintainable code

React Flow: robust graph editor for node-edge workflows

Zustand: lightweight global state management with minimal boilerplate

React Hook Form: efficient controlled forms for node configuration

This combination enables fast feature delivery while keeping architecture production-friendly.

4) Functional Coverage

4.1 Workflow Canvas

Implemented:

Drag-and-drop node creation from sidebar

Edge connections between nodes

Node and edge deletion

Node selection and edit panel integration

Canvas helpers (minimap, controls, background)

Basic validation constraints (start/end presence, cycle/disconnected checks, start-order warning)

4.2 Node Types and Config Forms

Each node has a dedicated configuration form:

Start Node: title + metadata key-value pairs

Task Node: title, description, assignee, due date, optional custom fields

Approval Node: title, approver role, auto-approve threshold

Automated Step Node: title, action from API + dynamic action params

End Node: end message + summary flag

This form-per-node pattern makes future node additions easy.

4.3 Mock API Layer

Implemented mock service:

GET /automations: returns automation actions and expected parameters

POST /simulate: validates workflow and returns step-by-step execution result

API logic is abstracted from UI, so real backend integration can be done later without major refactor.

4.4 Simulation / Sandbox

Implemented test panel:

Runs structural validation before simulation

Serializes current graph (nodes, edges)

Simulates execution path from start node

Displays execution timeline, statuses, timestamps, and errors

This validates not just UI behavior but workflow logic reasoning.

5) Architecture and Folder Strategy

The project is organized by responsibility:

api/ – mock network contracts and simulation logic

components/canvas/ – React Flow integration

components/forms/ – node-specific form components

components/panels/ – edit panel + simulation panel

components/sidebar/ – draggable node palette

hooks/ – reusable node/workflow actions

store/ – centralized workflow state

types/ – domain interfaces

utils/ – node defaults, labels, helper logic

This decomposition supports maintainability and scaling.

6) State Management Strategy

Zustand manages:

Multiple workflows

Active workflow

Selected node

Validation errors

Simulation status/results

Benefits:

Predictable updates

Minimal prop drilling

Easy feature extension (import/export, undo/redo, persistence)

7) Validation and Execution Logic

Validation rules currently include:

Start node required

End node required

Cycle detection

Disconnected node detection

Start node ordering warning

Simulation behavior:

Validates before run

Traverses graph from start node

Produces deterministic step timeline

Returns success/failure with errors

8) Key Engineering Decisions

Dedicated forms per node type instead of a giant generic form for clarity and scalability

Mock API abstraction to keep business flow independent of transport layer

Strong typing for node data to reduce runtime errors

Central store design to coordinate canvas, edit panel, and simulation panel reliably

9) Assumptions

No authentication required

No backend persistence required

Single-user in-memory session

Prototype prioritizes architecture and core flow over pixel-perfect styling

Simulation follows a simplified deterministic path

10) Limitations and Future Enhancements

Potential next steps:

JSON import/export

Undo/redo stack

Visual node-level error indicators

Auto-layout support

Node templates

Backend persistence and workflow versioning

11) Outcome

The prototype successfully demonstrates:

React Flow proficiency (custom nodes + edge management)

Scalable React architecture

Complex and dynamic form handling

Clean mock API integration

Functional workflow testing sandbox

Overall, this solution meets the case study objective by delivering a working, extensible HR workflow designer within prototype scope and timeline.